

CBeave Version 1

Entity-Relationship Diagram Guide

Current schema summary, complete field reference, use cases, and workflows

Executable source: `apps/api/prisma/schema.prisma`

Database: PostgreSQL with Prisma ORM

Updated: 2026-07-21

Scope note

This guide matches the current Version 1 Prisma schema. V1 includes local, Google, and Facebook login; accepted-only bid persistence; Live Arena participation; in-app notifications; focused user/category administration; and emergency auction cancellation. Password recovery, auction reporting, payments, shipping, external notification delivery, and advanced analytics are deferred.

How to use this guide

Each table section explains why the table exists, every stored field, important indexes, a practical use case, and the service workflow. Application-only rules are called out because a relational schema cannot enforce every cross-row or role condition.

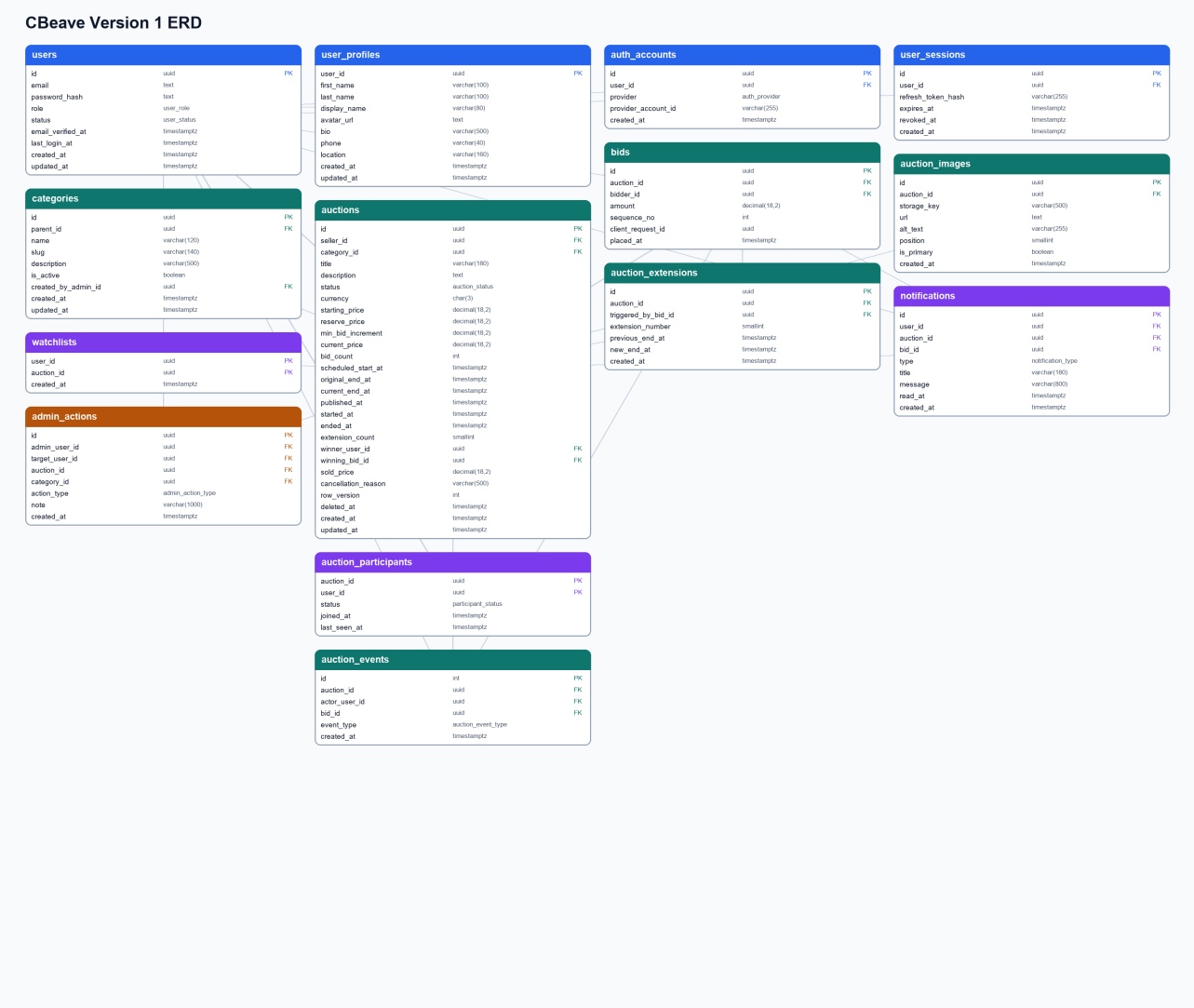
Domain	Tables	Responsibility
Identity and access	users, user_profiles, auth_accounts, user_sessions	Accounts, profiles, OAuth links, and refresh sessions.
Auction core	categories, auctions, bids, auction_images, auction_extensions, auction_events, auction_recycle, bids, images, extensions, and event history	Auction core lifecycle, bids, images, extensions, and event history.
Engagement	watchlists, auction_participants, notifications	Watchlists, Live Arena presence, and in-app alerts.
Governance	admin_actions	Focused administrator audit trail.

Core implementation rules

- Guest is not a stored role. USER and ADMIN are persisted roles.
- One USER may buy and sell but cannot bid on their own auction.
- Only accepted bids are stored; rejected requests create no bid row.
- Reserve price is private and reserve-met state is derived.
- Bid creation, aggregate price/count/version updates, and extensions are atomic.
- Category and admin target rules are validated by services.
- Notification and event JSON snapshots are not stored; reconnecting clients refetch authoritative state.

Relationship overview

Lines show table-level references. Exact keys and constraints are listed in each table section.



Source: docs/architecture/erd/v1/cbeave-erd-v1.dbml

Enum reference

Enum	Values	Purpose
user_role	USER, ADMIN	Separates marketplace and administration permissions.
user_status	ACTIVE, SUSPENDED, DEACTIVATED	Controls authentication and protected activity.
auth_provider	GOOGLE, FACEBOOK	External V1 login providers.
auction_status	DRAFT, SCHEDULED, ACTIVE, SOLD, UNSOLD, CANCELLED	Canonical persisted lifecycle.
participant_status	JOINED, LEFT	Current Live Arena participation state.
notification_type	OUTBID, AUCTION_WON, AUCTION_ENDED, AUCTION_CANCELLED	In-app alert categories.
auction_event_type	CREATED, PUBLISHED, STARTED, BID_PLACED, EXTENDED, ENDED, CANCELLED	Compact auction history event types.
admin_action_type	SUSPEND_USER, REACTIVATE_USER, CREATE_CATEGORY, UPDATE_CATEGORY, ACTIVATE_CATEGORY, DEACTIVATE_CATEGORY, CANCEL_AUCTION	Audited V1 admin actions.

Enums constrain stored values but services must still enforce valid lifecycle transitions and action-target combinations.

Identity and access

Tables in this domain

users

Stores the account identity, authorization role, account status, and authentication timestamps shared by local and social login.

Use case

A person may register with a password or arrive through Google or Facebook. All methods resolve to one users row, which is checked before protected operations.

Workflow

Create the user, create the profile, optionally create an OAuth account, then issue a refresh session. On every login, reject suspended or deactivated users and update last_login_at.

Fields

Field	Type	Rules	Meaning
id	uuid	PK, generated	Stable internal user identifier.
email	text	required, unique	Normalized login/contact email. The application should store lowercase form.
password_hash	text	optional	Secure password hash; null for social-only users.
role	user_role	required, USER	USER participates in auctions; ADMIN manages the platform.
status	user_status	required, ACTIVE	Blocks authentication and protected activity when suspended or deactivated.
email_verified_at	timestampz	optional	When the application accepted the email as verified.
last_login_at	timestampz	optional	Time of the most recent successful login.
created_at	timestampz	required, now	Account creation time.
updated_at	timestampz	required, auto	Last account-row update time.

Indexes and constraints: UNIQUE(email); INDEX(role, status)

user_profiles

Separates public/profile information from security-sensitive account fields.

Use case

Auction cards and Live Arena identify a bidder with `display_name` while the user may maintain optional biography, phone, location, and avatar data.

Workflow

Create one profile for a user during onboarding. Update it through the authenticated profile endpoint; deleting the user cascades to this row.

Fields

Field	Type	Rules	Meaning
<code>user_id</code>	<code>uuid</code>	PK, FK users	Also enforces exactly one profile row per user.
<code>first_name</code>	<code>varchar(100)</code>	required	Given or preferred first name.
<code>last_name</code>	<code>varchar(100)</code>	optional	Optional family name; supports single-name users.
<code>display_name</code>	<code>varchar(80)</code>	required	Public identity used in marketplace views.
<code>avatar_url</code>	<code>text</code>	optional	Current profile image URL.
<code>bio</code>	<code>varchar(500)</code>	optional	Short user biography.
<code>phone</code>	<code>varchar(40)</code>	optional	Profile contact phone; not required for V1 auth.
<code>location</code>	<code>varchar(160)</code>	optional	Free-form public or profile location.
<code>created_at</code>	<code>timestampz</code>	required, now	Profile creation time.
<code>updated_at</code>	<code>timestampz</code>	required, auto	Most recent profile update.

Indexes and constraints: INDEX(`display_name`)

auth_accounts

Links a CBeave user to an external Google or Facebook identity.

Use case

The same CBeave user can sign in with one Google identity and one Facebook identity without duplicating the user account.

Workflow

After OAuth callback verification, look up provider plus provider_account_id. Create a link only through a trusted onboarding/linking flow; never merge solely from an unverified email match.

Fields

Field	Type	Rules	Meaning
id	uuid	PK, generated	Internal OAuth-link identifier.
user_id	uuid	required, FK users	CBeave user that owns this external identity.
provider	auth_provider	required	GOOGLE or FACEBOOK.
provider_account_id	varchar(255)	required	Stable account identifier returned by the provider.
created_at	timestampz	required, now	When the provider was linked.

Indexes and constraints: UNIQUE(provider, provider_account_id); UNIQUE(user_id, provider)

user_sessions

Stores revocable refresh sessions without storing raw refresh tokens.

Use case

A user can remain signed in, refresh an access token, and log out by revoking the matching session.

Workflow

Generate a refresh token, store only its hash, and set expiry. Refresh validates hash, expiry, revocation, and user status. Logout sets revoked_at; cleanup removes expired or revoked rows later.

Fields

Field	Type	Rules	Meaning
id	uuid	PK, generated	Session identifier.
user_id	uuid	required, FK users	Owner of the session.
refresh_token_hash	varchar(255)	required, unique	One-way hash used to validate a refresh token.
expires_at	timestamptz	required	Hard refresh-session expiration.
revoked_at	timestamptz	optional	Set on logout or forced revocation.
created_at	timestamptz	required, now	Session issue time.

Indexes and constraints: INDEX(user_id, expires_at)

Auction core

Tables in this domain

categories

Provides a two-level-capable, administrator-managed auction taxonomy.

Use case

An administrator creates Bicycles and optional child categories such as Mountain Bikes. Sellers select an active category when publishing.

Workflow

Seed initial rows or create them through admin UI. Use `parent_id` for hierarchy and `slug` for URLs. Deactivate referenced categories instead of deleting them.

Fields

Field	Type	Rules	Meaning
id	uuid	PK, generated	Category identifier.
parent_id	uuid	optional, self FK	Parent category; null for top-level categories.
name	varchar(120)	required	Human-readable category name.
slug	varchar(140)	required, unique	Stable URL-safe category identifier.
description	varchar(500)	optional	Short category explanation.
is_active	boolean	required, true	Controls new use and public category availability without deletion.
created_by_admin_id	uuid	optional, FK users	Administrator that created the row; null supports seed/system data.
created_at	timestampz	required, now	Creation time.
updated_at	timestampz	required, auto	Most recent category update.

Indexes and constraints: `UNIQUE(slug)`; `INDEX(parent_id, is_active)`; `INDEX(is_active)`

auctions

Authoritative current aggregate for ownership, lifecycle, pricing, schedule, result, and optimistic concurrency.

Use case

A seller drafts and schedules a bicycle auction. The scheduler starts it, accepted bids update its current values, and the ending job stores Sold or Unsold result.

Workflow

Create Draft, validate and Publish to Scheduled/Active, atomically update price/count/version for each accepted bid, apply extensions, then store terminal result. Use deleted_at only for soft deletion where allowed.

Fields

Field	Type	Rules	Meaning
id	uuid	PK, generated	Auction identifier.
seller_id	uuid	required, FK users	USER that owns and lists the auction.
category_id	uuid	required, FK categories	Selected active category.
title	varchar(180)	required	Public auction title.
description	text	required	Full item description.
status	auction_status	required, DRAFT	Persisted lifecycle state.
currency	char(3)	required, USD	ISO-style currency code used by all monetary values.
starting_price	decimal(18,2)	required	Opening price before accepted bids.
reserve_price	decimal(18,2)	optional, private	Hidden minimum sale price; never sent to buyers.
min_bid_increment	decimal(18,2)	required	Minimum increase required for the next bid.
current_price	decimal(18,2)	required	Fast current accepted price; maintained transactionally.
bid_count	int	required, 0	Accepted-bid count and simple Hot Auctions ranking input.
scheduled_start_at	timestampz	optional	Planned start after publication.
original_end_at	timestampz	optional	End time before anti-sniping extensions.
current_end_at	timestampz	optional	Authoritative current deadline.
published_at	timestampz	optional	When Publish succeeded.
started_at	timestampz	optional	Actual transition to Active.
ended_at	timestampz	optional	Actual terminal transition time.
extension_count	smallint	required, 0	Number of anti-sniping extensions; application maximum is five.
winner_user_id	uuid	optional, FK users	Winning user for a Sold auction.
winning_bid_id	uuid	optional, unique FK bids	Accepted bid that produced the sale result.
sold_price	decimal(18,2)	optional	Final sale price frozen at completion.
cancellation_reason	varchar(500)	optional	Seller/admin/system cancellation explanation.
row_version	int	required, 1	Optimistic concurrency counter updated with aggregate changes.
deleted_at	timestampz	optional	Soft-delete marker; public queries exclude non-null rows.
created_at	timestampz	required, now	Draft creation time.
updated_at	timestampz	required, auto	Most recent aggregate update.

Indexes and constraints: UNIQUE(winning_bid_id); INDEX(status, scheduled_start_at); INDEX(status, current_end_at); INDEX(category_id, status); INDEX(seller_id, status); INDEX(status, bid_count)

bids

Stores the immutable chronological history of accepted bids only.

Use case

A bidder retries after a slow connection. `client_request_id` prevents duplication, while `sequence_no` provides stable per-auction ordering.

Workflow

Validate eligibility and amount inside a transaction. If invalid, return an error and create no row. If valid, insert the bid, update the auction aggregate, record any extension/event, and broadcast after commit.

Fields

Field	Type	Rules	Meaning
id	uuid	PK, generated	Accepted bid identifier.
auction_id	uuid	required, FK auctions	Auction receiving the bid.
bidder_id	uuid	required, FK users	USER that placed the accepted bid.
amount	decimal(18,2)	required	Accepted monetary amount.
sequence_no	int	required	Monotonic accepted-bid number within one auction.
client_request_id	uuid	required, unique	Idempotency key generated by the client.
placed_at	timestampz	required, now	Server-recorded accepted time.

Indexes and constraints: UNIQUE(client_request_id); UNIQUE(auction_id, sequence_no); INDEX(auction_id, amount); INDEX(bidder_id, placed_at)

auction_images

Stores external object metadata and display ordering for auction images.

Use case

A bicycle auction has a cover image plus detail images ordered for cards and the auction page.

Workflow

Upload to external storage, persist stable storage_key and display URL, assign unique positions, and choose one primary image through application validation.

Fields

Field	Type	Rules	Meaning
id	uuid	PK, generated	Image metadata identifier.
auction_id	uuid	required, FK auctions	Owning auction.
storage_key	varchar(500)	required, unique	Stable object-storage identifier used for replacement/deletion.
url	text	required	Display URL, which may change with CDN configuration.
alt_text	varchar(255)	optional	Accessible description of the image.
position	smallint	required, 0	Ordering within the auction gallery.
is_primary	boolean	required, false	Cover image used by cards and initial detail view.
created_at	timestampz	required, now	Metadata creation time.

Indexes and constraints: UNIQUE(storage_key); UNIQUE(auction_id, position)

auction_extensions

Audits every anti-sniping deadline change and the accepted bid that caused it.

Use case

A bid arrives within the final two minutes; the service extends the deadline by two minutes and displays extension 3 in the Live Arena.

Workflow

Inside the bid transaction, check the time window and maximum count, update `auctions.current_end_at/count`, and insert one extension row. A bid can trigger at most one extension.

Fields

Field	Type	Rules	Meaning
id	uuid	PK, generated	Extension record identifier.
auction_id	uuid	required, FK auctions	Extended auction.
triggered_by_bid_id	uuid	required, unique FK bids	Accepted bid that triggered this extension.
extension_number	smallint	required	1-based extension number for this auction.
previous_end_at	timestampz	required	Deadline before extension.
new_end_at	timestampz	required	Deadline after extension.
created_at	timestampz	required, now	Extension creation time.

Indexes and constraints: UNIQUE(triggered_by_bid_id); UNIQUE(auction_id, extension_number)

auction_events

Stores a compact ordered audit/event stream for important auction lifecycle facts.

Use case

A reconnecting client or support view can see that an auction started, received a bid, was extended, and ended in a stable order.

Workflow

Insert an event when a domain transition commits. Use actor_user_id for user/admin actions, null for scheduler/system events, and optional bid_id for bid-related facts. Refetch authoritative state rather than relying on JSON snapshots.

Fields

Field	Type	Rules	Meaning
id	int	PK, auto-increment	Compact global event sequence for V1.
auction_id	uuid	required, FK auctions	Auction whose history contains the event.
actor_user_id	uuid	optional, FK users	User/admin actor; null for automated system events.
bid_id	uuid	optional, FK bids	Accepted bid associated with BID_PLACED/EXTENDED or another bid-related event.
event_type	auction_event_type	required	CREATED, PUBLISHED, STARTED, BID_PLACED, EXTENDED, ENDED, or CANCELLED.
created_at	timestampz	required, now	Event occurrence/record time.

Indexes and constraints: INDEX(auction_id, id); INDEX(event_type, created_at)

Engagement

Tables in this domain

watchlists

Records which public auctions a user has saved.

Use case

A user saves a Starting Soon auction and later opens a personal list with current countdown and status.

Workflow

Insert the composite user/auction key on Watch; delete it on Unwatch. The composite primary key prevents duplicates.

Fields

Field	Type	Rules	Meaning
user_id	uuid	PK part, FK users	User who watches the auction.
auction_id	uuid	PK part, FK auctions	Watched auction.
created_at	timestampz	required, now	When the auction was added.

Indexes and constraints: PRIMARY KEY(user_id, auction_id); INDEX(auction_id, created_at)

auction_participants

Persists one current Live Arena participation state per user and auction.

Use case

The lobby displays how many authenticated users are currently joined and records when each was last seen.

Workflow

Upsert JOINED when a user enters, update last_seen_at on heartbeat, and set LEFT on an explicit/timeout departure. RSVP is not part of V1.

Fields

Field	Type	Rules	Meaning
auction_id	uuid	PK part, FK auctions	Live Arena auction.
user_id	uuid	PK part, FK users	Participating user.
status	participant_status	required, JOINED	Current JOINED or LEFT state.
joined_at	timestampz	required, now	First/current persisted join time.
last_seen_at	timestampz	optional	Latest heartbeat/activity time.

Indexes and constraints: PRIMARY KEY(auction_id, user_id); INDEX(auction_id, status)

notifications

Stores user-specific in-app alerts for auction outcomes and bidding changes.

Use case

The former leader receives an Outbid alert; the winner receives Auction Won; sellers receive Auction Ended; affected users may receive Auction Cancelled.

Workflow

Create rows after the authoritative transaction commits, broadcast notification:created to each recipient, list newest/unread rows, and set read_at when opened.

Fields

Field	Type	Rules	Meaning
id	uuid	PK, generated	Notification identifier.
user_id	uuid	required, FK users	Recipient.
auction_id	uuid	required, FK auctions	Auction context for every V1 notification.
bid_id	uuid	optional, FK bids	Related accepted bid for bid-driven notifications; null for result/cancellation when appropriate.
type	notification_type	required	OUTBID, AUCTION_WON, AUCTION_ENDED, or AUCTION_CANCELLED.
title	varchar(180)	required	Short in-app heading.
message	varchar(800)	required	Human-readable notification body.
read_at	timestampz	optional	Null means unread; timestamp means opened/read.
created_at	timestampz	required, now	Creation/delivery ordering time.

Indexes and constraints: INDEX(user_id, read_at, created_at)

Governance

Tables in this domain

admin_actions

Provides an immutable audit trail for the deliberately small Version 1 administration surface.

Use case

An administrator suspends a user, reactivates a category, or cancels an inappropriate auction, and the action can later be reviewed.

Workflow

Authorize ADMIN, validate that exactly the correct target is supplied for the action type, perform the state change and insert the audit row in one transaction where practical.

Fields

Field	Type	Rules	Meaning
id	uuid	PK, generated	Audit action identifier.
admin_user_id	uuid	required, FK users	Administrator who performed the action.
target_user_id	uuid	optional, FK users	Required for suspend/reactivate user actions.
auction_id	uuid	optional, FK auctions	Required for cancel auction action.
category_id	uuid	optional, FK categories	Required for category actions.
action_type	admin_action_type	required	Audited operation.
note	varchar(1000)	optional	Human explanation or internal audit note.
created_at	timestampz	required, now	Action time.

Indexes and constraints: INDEX(admin_user_id, created_at); INDEX(action_type, created_at)

Relationship reference

Foreign keys preserve identity and referential integrity. Application services add ownership, role, same-auction, and exactly-one-target rules that cannot be expressed by these keys alone.

From	To	Purpose
user_profiles.user_id	users.id	one-to-one profile
auth_accounts.user_id	users.id	OAuth identities
user_sessions.user_id	users.id	refresh sessions
categories.parent_id	categories.id	category hierarchy
categories.created_by_admin_id	users.id	creator audit
auctions.seller_id	users.id	seller ownership
auctions.category_id	categories.id	classification
auctions.winner_user_id	users.id	sale winner
auctions.winning_bid_id	bids.id	one winning bid
bids.auction_id	auctions.id	accepted bid history
bids.bidder_id	users.id	bidder history
auction_images.auction_id	auctions.id	gallery
watchlists.user_id / auction_id	users.id / auctions.id	saved auctions
auction_participants.user_id / auction_id	users.id / auctions.id	Live Arena presence
auction_extensions.auction_id / triggered_by_bid_id	auctions.id / bids.id	anti-sniping audit
notifications.user_id / auction_id / bid_id	users.id / auctions.id / bids.id	recipient and context
admin_actions targets	users / auctions / categories	typed admin audit target
auction_events auction/actor/bid	auctions / users / bids	ordered history context

End-to-end workflows

Local registration and session

Validate names/email/password -> create users and user_profiles -> hash password -> create user_sessions -> return access and refresh credentials.

Google or Facebook login

Verify provider callback -> find auth_accounts by provider and provider_account_id -> create/link user through a trusted flow when absent -> reject unusable missing-email onboarding -> create user_sessions.

Publish and schedule

Create DRAFT -> add auction_images -> validate pricing/category/schedule -> Publish to SCHEDULED or ACTIVE -> insert auction_events.PUBLISHED -> scheduler inserts STARTED at activation.

Accepted bid

Validate Active state, role, ownership, amount, deadline, and idempotency -> insert bids -> atomically update current_price, bid_count, row_version -> optionally insert auction_extensions -> insert auction_events -> commit -> broadcast.

Reserve broadcast

Derive reserveMet from private reserve_price and accepted current price -> include only the boolean in auction responses/broadcasts -> do not store reserve_met_at or create a notification.

Auction completion

Lock/check due auction -> select accepted highest bid -> compare private reserve -> set SOLD with winner/winning bid/sold price or UNSOLD -> set ended_at -> insert ENDED event -> create result notifications.

Administration

Authorize ADMIN -> validate target for action type -> change users/categories/auction state -> insert admin_actions with actor, target, optional note, and timestamp.

Reconnect

Use auction_events only as ordered facts -> refetch auctions, recent bids, extensions, and participant state -> rebuild Live Arena from authoritative rows.

Deferred beyond Version 1

Password reset and external email delivery; IP/device session metadata; RSVP; user-submitted auction reports and report resolution; curated featured auctions; payment/order/shipping domains; messaging/reviews; multi-channel delivery; and advanced analytics.

Synchronization rule

When persistent data changes, update schema.prisma and its migration, this guide and V1 DBML, the SRS/ADR when behavior changes, the traceability matrix, and the repository changelog in the same pull request.